

PRM3

Théorie : Leçon 10

Java :
Les tableaux

Les tableaux

Un **tableau** est une **collection indexée** (indicée) **d'éléments du même type**.

Les éléments peuvent être quelconques : entiers, caractères, personnes, adresses, ...

On **accède à un élément** du tableau (une composante) à l'aide de son indice (accès direct, accès indexé).

L'indice doit être compris entre 0 et $\text{taille} - 1$

Le type `eu.epfc.prm3.Array`

Des collections indexées existent déjà dans la librairie Java mais pour des raisons pédagogiques, nous avons créé le type `Array`.

Il est disponible dans le fichier `Array.jar` fourni que vous importerez dans vos projets.

```

import eu.epfc.prm3.Array;

public class Demo {
    public static void main(String[] args) {
        Array<Integer> tab = new Array<>(1, 2, 3, 4, 5);
        System.out.println(tab);           // [1, 2, 3, 4, 5]
        System.out.println(tab.get(3));    // 4
        tab.set(2, 10);
        System.out.println(tab);           // [1, 2, 10, 4, 5]
        tab.add(0);
        System.out.println(tab);           // [1, 2, 10, 4, 5, 0]
        System.out.println(tab.size());    // 6
    }
}

```

Création d'un tableau

Création d'un tableau d'entiers pré-rempli :

```
Array<Integer> tab1 = new Array<>(2, 7, 1, 3, 2);
```

2	7	1	3	2
0	1	2	3	4

Création d'un tableau de réels pré-rempli :

```
Array<Double> tab2 = new Array<>(1.3, 2.5, 7.0);
```

1.3	2.5	7.0
0	1	2

Création d'un tableau d'entiers vide :

```
Array<Integer> tab3 = new Array<>();
```

Le paramètre de type (le type des éléments du tableau) peut être choisi parmi Integer, Double, Character, Boolean, ...

Accès aux éléments d'un tableau

```
Array<Integer> tab = new Array<>(2, 7, 1, 3, 2);
```

2	7	1	3	2
0	1	2	3	4

La méthode `get(int index)` appliquée sur un tableau permet d'obtenir un élément via son indice :

```
int val = tab.get(1); //val vaut 7
```

La méthode `set(int index, Type val)` change la valeur d'un élément :

```
tab.set(3, 8); //l'élément d'indice 3 prend la valeur 8
```

2	7	1	8	2
0	1	2	3	4

Attention, l'indice doit toujours rester dans les bornes (entre 0 et taille - 1) sinon une exception est déclenchée et le programme s'arrête.

Parcours d'un tableau

tab	2	7	1	3	2
	0	1	2	3	4

Parcours via les indices (remarquez que la méthode `size()` donne la taille) :

```
for(int i = 0; i < tab.size(); ++i)
    System.out.println(tab.get(i));
```

Parcours via la boucle *for-each* (uniquement pour la lecture) :

```
for(int val : tab) // pour chaque valeur du tableau
    System.out.println(val);
```

Affichage d'un tableau :

```
System.out.println(tab); // [2, 7, 1, 3, 2]
```

Ajout et suppression d'éléments (I)

tab

2	7	1	3	2
0	1	2	3	4

La méthode `add(Type val)` ajoute un élément à la fin du tableau :

`tab.add(3);`

tab

2	7	1	3	2	3
0	1	2	3	4	5

La méthode `extend(int nbVal, Type val)` ajoute plusieurs éléments identiques à la fin du tableau :

`tab.extend(3, 0);`

tab

2	7	1	3	2	3	0	0	0
0	1	2	3	4	5	6	7	8

Ajout et suppression d'éléments (2)

tab

2	7	1	3	2
0	1	2	3	4

La méthode `reduceTo(int newSize)` réduit la taille du tableau en supprimant les derniers éléments :

`tab.reduceTo(2);`

tab

2	7
0	1

Tableaux et fonctions

Les tableaux sont des objets muables (ils peuvent changer).

Cela implique, notamment, qu'au contraire des types vus précédemment (primitifs et SeqInt), quand l'on passe un tableau à une fonction, celle-ci peut le modifier.

```

public class Exemple {
    public static void multiplieParDeux(Array<Integer> tab){
        for(int i = 0; i < tab.size(); ++i)
            tab.set(i, tab.get(i) * 2);
    }

    public static void main(String[] args) {
        Array<Integer> tab = new Array<>(2, 7, 1, 3, 2);

        System.out.println(tab); // [2, 7, 1, 3, 2]

        multiplieParDeux(tab);

        System.out.println(tab); // [4, 14, 2, 6, 4]
    }
}

```

Résumé des opérations sur les tableaux

Méthode	Description
<code>int size()</code>	Renvoie la taille du tableau.
<code>Type get(int index)</code>	Renvoie la valeur située à l'index donné.
<code>Type set(int index, Type val)</code>	Remplace l'élément situé à l'indice donné par la valeur donnée. Renvoie l'ancienne valeur.
<code>boolean add(Type val)</code>	Ajoute une valeur à la fin du tableau. Renvoie toujours vrai.
<code>void extend(int nbVal, Type val)</code>	Ajoute plusieurs valeurs identiques à la fin du tableau
<code>void reduceTo(int newSize)</code>	Réduit la taille du tableau vers la taille donnée en supprimant les derniers éléments.